

SIMCOMP

Sistema de Comparendos de Tránsito

Arquitectura de microservicios REST para gestión de comparendos, autenticación, frontend React y despliegue en Docker o Vagrant + Ansible.

Universidad Autónoma de Occidente
Ingeniería de Datos e Inteligencia Artificial
Redes e Infraestructuras · 11 de marzo de 2026

Resumen ejecutivo

6

microservicios backend

5

clusters PostgreSQL

3

VMs en Vagrant

JWT

seguridad y sesiones

Autores

Deyton Riascos Ortiz — 2246208

Samuel Izquierdo Bonilla — 2246993

Mauricio Taborda Gongora — 2246998

CONTEXTO

Problema, alcance y principios de diseño

El proyecto organiza la gestión de tránsito con servicios desacoplados, datos propios y validación entre dominios.

PROPÓSITO

Qué resuelve

Centraliza personas, automotores, infracciones, autenticación y comparendos en una solución ejecutable localmente o en 3 VMs provisionadas.

DISEÑO

Restricciones clave

Cada microservicio es dueño de sus datos. Las referencias entre servicios se manejan con UUID y APIs REST síncronas; no hay llaves foráneas cruzadas.

STACK

Tecnologías base

Node.js 22 · Express 4 · PostgreSQL 14 · Vagrant · Ansible · Swagger/OpenAPI · Frontend React 19 + TypeScript + Vite.

- Comunicación entre servicios exclusivamente vía REST/HTTP.
- Valores monetarios modelados con DECIMAL(12,2).
- Cada servicio expone health check y documentación Swagger.
- La consistencia se garantiza a nivel de servicio y reglas de negocio.

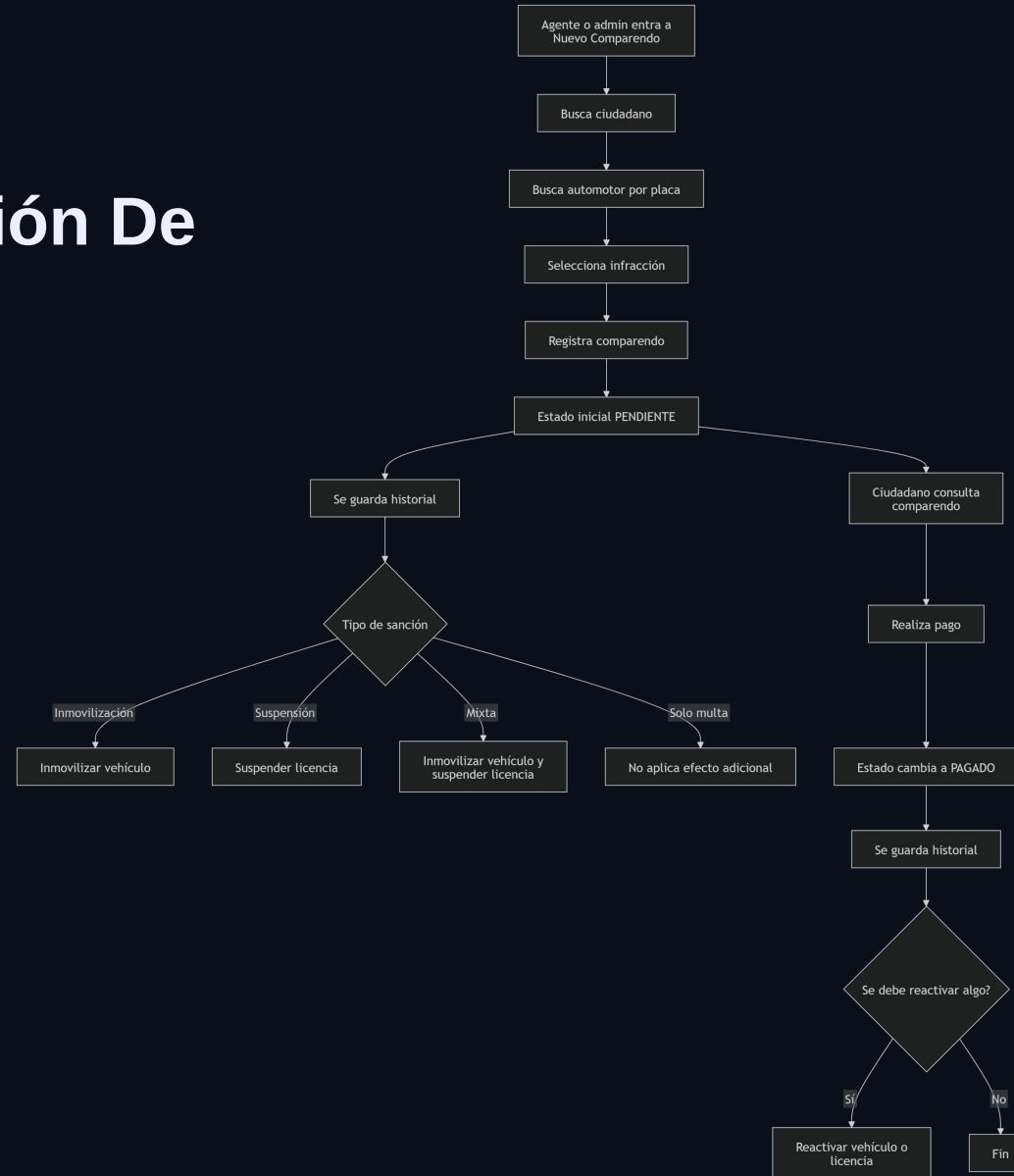
Topología general de despliegue

Producción académica sobre 3 VMs: DNS, API + bases de datos y Web/Gateway; además puede ejecutarse en Docker para entorno local.



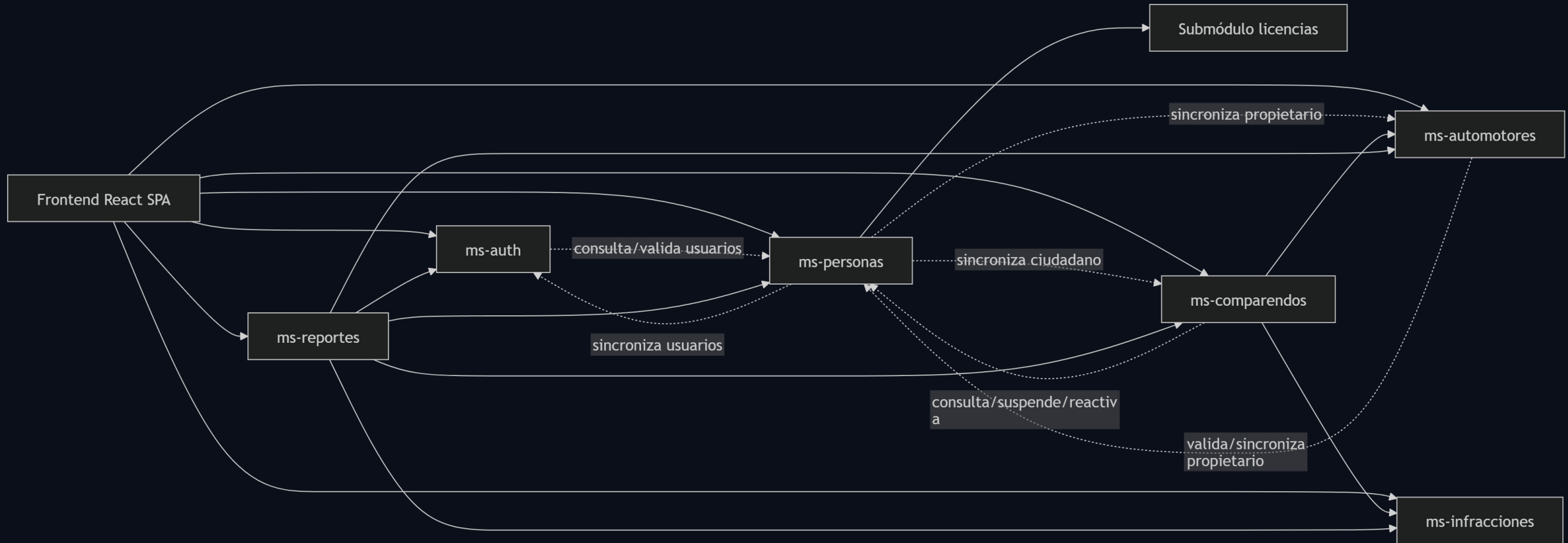
Proceso Completo De Gestión De Comparendos

Describe cómo un agente registra un comparendo, las validaciones realizadas y los posibles estados y sanciones. Incluye el flujo desde la creación hasta el pago y cierre del proceso.



Validación De Usuarios Y Control Por Roles

Muestra el proceso de login, validación de credenciales y uso de tokens (JWT). Define cómo se protege el acceso a los módulos según el rol del usuario.



Estructura Lógica Del Sistema

Organiza la aplicación en capas (presentación, seguridad, lógica de negocio y datos), facilitando mantenimiento, escalabilidad y separación de responsabilidades.

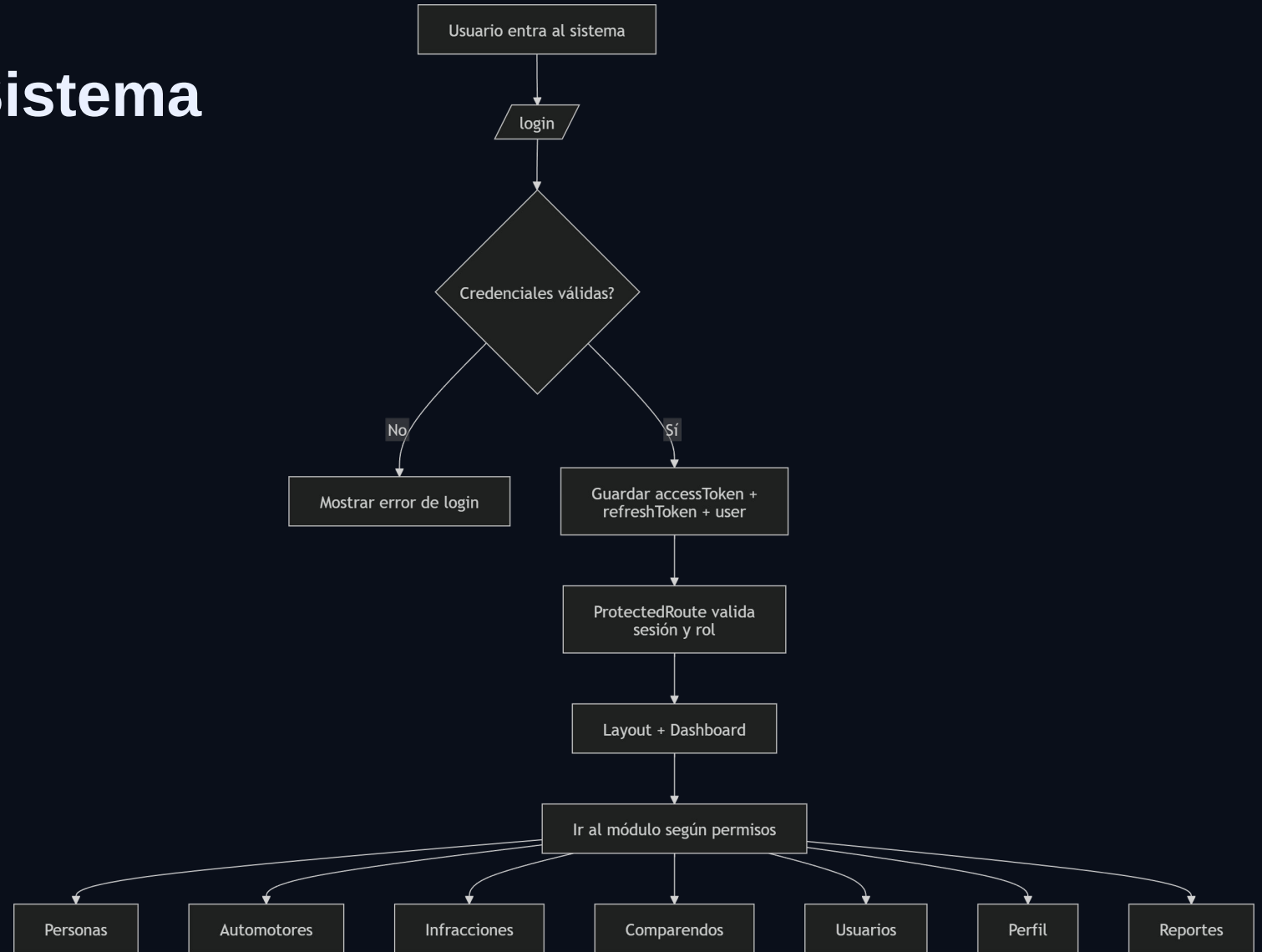
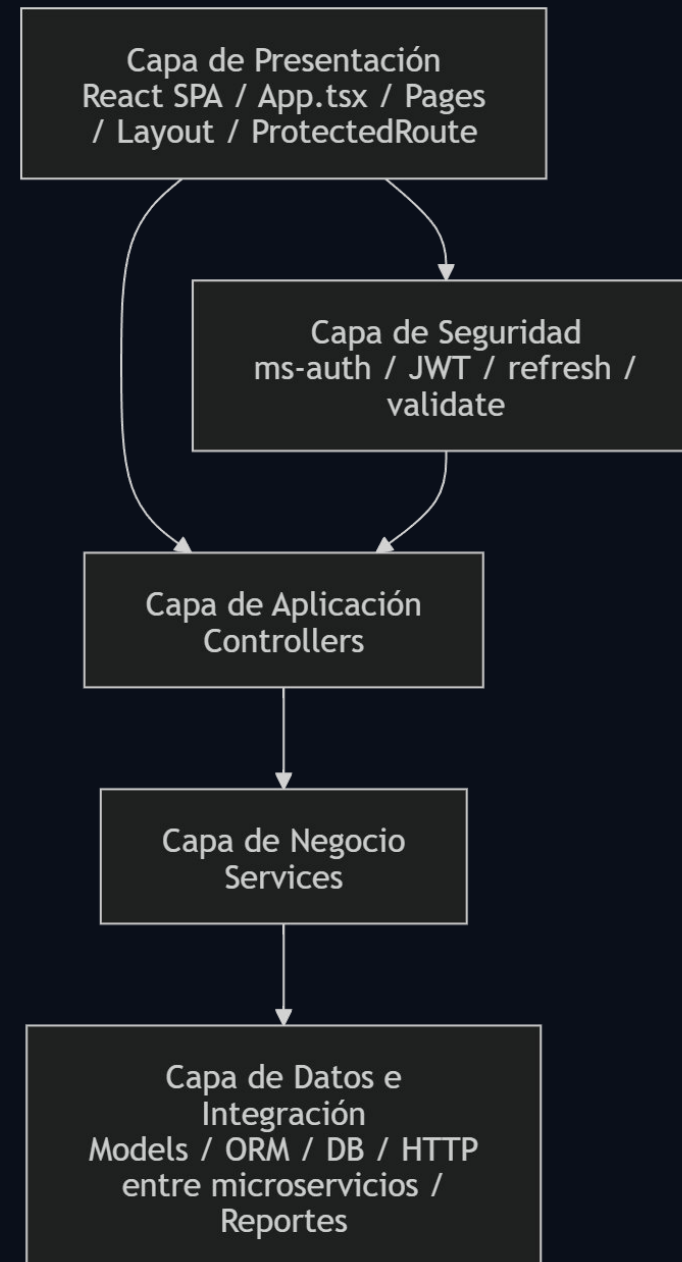


DIAGRAMA POR CAPAS

Comunicación E Integración Del Sistema

Explica cómo los microservicios interactúan entre sí mediante APIs, compartiendo información de usuarios, vehículos, infracciones y comparendos.



Seis microservicios y sus puertos correctos

Los servicios corren en 8001–8005.

8001

ms-auth-service

Usuarios, login, refresh token, validación JWT y roles del sistema.

8002

ms-personas

Ciudadanos, datos personales y licencias de conducción.

8003

ms-automotores

Registro técnico del vehículo y asociación con propietario.

8004

ms-infracciones

Catálogo oficial de infracciones y multas vigentes.

8005

ms-comparendos

Core del negocio: registra comparendos y maneja el ciclo de vida.

8006

ms-reportes

Exportación e importación de reportes en formatos xlsx, csv y pdf.

El servicio de comparendos depende lógicamente de personas, automotores e infracciones, y además la plataforma incorpora autenticación como quinto microservicio de backend.

Propiedad de datos y clusters PostgreSQL

Se aísla completamente el almacenamiento: un servicio, una base de datos, un puerto y un cluster independiente.

Servicio	Puerto API	DB	Puerto DB	Cluster
ms-auth-service	8001	auth_db	5432	auth
ms-personas	8002	personas_db	5433	personas
ms-automotores	8003	automotores_db	5434	automotores
ms-infracciones	8004	infracciones_db	5435	infracciones
ms-comparendos	8005	comparendos_db	5436	comparendos
ms-reportes	8006	reportes_db		reportes

INTEGRIDAD

Regla de integración

No existen claves foráneas físicas entre bases de datos distintas; las validaciones se resuelven vía HTTP/REST.

CONSISTENCIA

Modelo de referencia

Los identificadores entre servicios son UUID. Los montos monetarios se gestionan como DECIMAL(12,2).

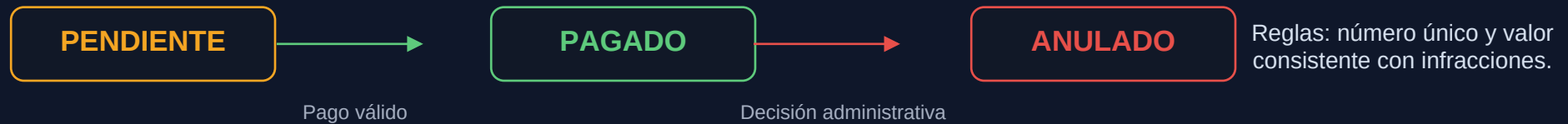
Beneficio: aislamiento, trazabilidad y facilidad de mantenimiento por dominio.

Creación y ciclo de vida del comparendo

El valor del sistema está en orquestar validaciones externas y mantener una máquina de estados controlada.



Máquina de estados base



React 19, rutas protegidas y flujo JWT

La SPA administra el acceso, consume los microservicios y delega la validación central al servicio de autenticación.

SPA

Frontend administrativo

React 19 + TypeScript + Vite + TailwindCSS +
React Query + React Hook Form.

Módulos principales: Login, Dashboard,
Personas, Automotores, Infracciones,
Comparendos y Usuarios.

Flujo JWT

- Login y emisión de tokens
- Access token en memoria del cliente
- Refresh token para renovar sesión
- Gateway valida JWT antes de cada API
- Si el refresh falla, vuelve al login

Roles

admin
agente
supervisor
ciudadano

Cuatro formas de ejecutar el proyecto

El proyecto completo puede correr en local con Docker Compose, o full Docker Composer, como entorno provisionado con Vagrant + Ansible o como entorno provisionado con vagrant + Ansible + Docker Compose y Swarm.

LOCAL

Modo 1 · Docker local

Uso rápido para desarrollo y demostración.

- `docker compose up --build -d`
- Frontend en `http://localhost`
- El ecosistema simula la red de producción.
- Útil para pruebas funcionales y desarrollo integrado.

PROVISIONADO VAGRANT + ANSIBLE

Modo 2 · Vagrant + Ansible

Entorno académico con 3 VMs.

- `vagrant up`
- DNS local + API + Web/Gateway
- PM2 para procesos Node.js
- Nginx para servir la SPA y enrutar APIs con JWT.

DOCKER COMPOSE COMPLETO (FULL STACK)

Modo 3 · Docker Compose completo (Full Stack)

Entorno académico con Docker.

- `docker compose -f docker-compose.full.yml up -d`
- Incluye frontend + backend + bases de datos
- Simula un entorno más cercano a producción
- Comunicación entre microservicios por red interna
- Útil para pruebas integrales del sistema

PROVISIONADO VAGRANT + ANSIBLE + DOCKER SWARM

Modo 4 · Vagrant + Ansible + Docker Swarm

Entorno académico con 3 VMs + Docker Swarm.

- `vagrant up` + aprovisionamiento con Ansible
- Inicialización de clúster con `docker swarm init`
- Despliegue con `docker stack deploy`
- Balanceo de carga (HAProxy)
- Escalado de servicios (replicas)
- Ideal para pruebas de carga y alta disponibilidad

URLs demo: `simcomp.co` · `api.simcomp.co:8001/api/docs` · `api.simcomp.co:8002/api/health` · ... · `api.simcomp.co:8006/api/health`

Secuencia sugerida para la sustentación

Una exposición sólida muestra el recorrido completo desde autenticación hasta el pago del comparendo.

01 Ingresar al login y validar roles del sistema.

04 Consultar automotor y verificar placa / propietario.

02 Mostrar dashboard y navegación por módulos.

05 Seleccionar infracción y crear comparendo.

03 Consultar o registrar persona y su licencia.

06 Consultar detalle, pagar o anular y revisar estado.

07 Consultar detalle, pagar o anular y revisar estado.

Tip: si preguntas por puertos del backend, la respuesta correcta es 8001, 8002, 8003, 8004, 8005 y 8006.

Arquitectura por capas de SIMCOMP

01

Acceso: navegador + DNS simcomp.co

04

Seguridad: ms-auth-service

02

Presentación: React SPA

05

Negocio: personas, autos, infracciones, comparendos y reportes

03

Gateway web: Nginx + JWT

06

Datos: PostgreSQL 14 / 5 clusters

07

Operación: PM2 + Vagrant + Ansible

Infraestructura : 3 VMs, BIND9, Nginx, React, Node.js 22, PM2 y PostgreSQL 14. Docker Compose.

SIMCOMP

Conclusión

El proyecto demuestra una arquitectura distribuida, ejecutable y coherente con principios de dominio, seguridad y despliegue en infraestructura virtualizada.

**6 microservicios backend
reales**

**Puertos correctos:
8001–8006**

**Frontend React + JWT +
Gateway**

**Docker local y Vagrant en 3
VMs**

Q&A

Gracias

¿Preguntas?